Kovex: Graph-Aware, Per-Node Coordination for Multi-Agent LLM Coding Systems

Ibrahem Alkurdi

ibrahem.alkurdi@gmail.com

May 8, 2026

Abstract

Multi-agent LLM coding systems that operate concurrently on shared codebases face a systematic failure mode that neither message-passing coordination nor text-level concurrency control can address: *semantic conflicts*, arising when one agent's write to a function signature or interface boundary invalidates assumptions held by a dependent agent. We present **Kovex**, a coordination substrate that indexes a live codebase into a Neo4j property graph, exposes it to agents via a Model Context Protocol server, and notifies registered dependents via bounded breadth-first search over the call graph when a write is committed. We evaluate Kovex on 14 tasks drawn from the httpx codebase, spanning three dependency distances ($d \in \{0, 1, 2\}$) and two false-negative control tasks at $d = 3$. Kovex correctly issues notifications for all 12 tasks at $d \leq 2$ (0% missed), produces zero notifications for $d = 3$ tasks (14.3% false-negative rate at $k = 2$), and delivers all directives in under 82 ms (mean 45.6 ms). Baseline agents with no coordination receive zero notifications and exhibit undetected conflicts across all 14 tasks.

1 Introduction

The deployment of large language models as collaborative coding agents has advanced rapidly, yet a fundamental coordination problem has emerged: when two agents operate concurrently on a shared codebase, one agent's modification to a function signature or type definition can silently invalidate the code being generated by the other. CooperBench [11] quantifies this as the *curse of coordination*, reporting a 30% lower success rate for collaborative agent pairs compared to single-agent execution.

CodeCRDT [18] demonstrates that text-level concurrency control, while guaranteeing syntactic convergence, still produces a 5–10% semantic conflict rate. Existing frameworks resolve concurrency by eliminating it (MetaGPT [9], ChatDev [19]) or by deferring to global test re-execution (AgentForge [14]).

We identify the root cause as a missing representation layer. No system maintains a persistent, queryable model of the codebase's dependency structure that agents can query and that can route write notifications to only those agents whose generated code is actually affected. Without this layer, the system cannot determine which agents need to re-verify after a given write, and verification is either absent or exhaustive.

We contribute **Kovex**, a coordination substrate comprising: (1) a Neo4j property graph indexed from live source code; (2) a Model Context Protocol (MCP) server through which agents register read dependencies on named graph nodes; and (3) a write-triggered bounded BFS notification algorithm that delivers targeted re-verification directives to registered dependents within a configurable k-hop call-graph radius. Our central research question is:

Does graph-aware, per-node read/write coordination with write-triggered targeted verification reduce cross-file semantic conflict rates in multi-agent coding systems, relative to uncoordinated baselines?

We evaluate Kovex on 14 controlled tasks against the httpx Python HTTP library [17], demonstrating that Kovex correctly detects and routes notifications for all scenarios within the $k = 2$ coordination radius, with a mean notification latency of 45.6 ms.

2 Related Work

2.1 Multi-Agent LLM Systems for Software Engineering

MetaGPT [9] encodes Standardized Operating Procedures into prompt sequences, assigning specialized roles that hand off work through structured artifacts. ChatDev [19] implements a similar role hierarchy through natural-language dialogue. AutoGen [24] generalizes this into a flexible multi-agent conversation framework. All three enforce coordination sequentially; no system maintains a runtime dependency model of the codebase. SWE-agent [26] focuses on single-agent performance on SWE-bench [10];

SWE-EVO [22] extends evaluation to long-horizon tasks requiring coordinated modifications across an average of 21 files, where the strongest agents achieve only 21% resolution.

CooperBench [11] provides the most direct empirical characterization of the coordination problem. Across 652 collaborative coding tasks, agent pairs achieve 30% lower success rates than single agents, with communication reducing syntactic merge conflicts but failing to prevent semantic conflicts arising from interface incompatibilities. AgentForge [14] advances the state of the art by binding verification to each state transition via sandboxed execution, achieving 40.0% on SWE-bench Lite. However, its verification is global and cannot determine from the codebase's dependency structure which agents need to re-verify.

2.2 Graph-Based Code Intelligence

Ferrante et al. [6] introduced the Program Dependence Graph, unifying control and data dependencies for each program operation. Yamaguchi et al. [25] unified the AST, CFG, and PDG into the Code Property Graph stored in a graph database and queryable for vulnerability detection—demonstrating that code structure can serve as a first-class database artifact rather than a transient in-memory representation. ACER [3] provides a language-agnostic pipeline using Tree-sitter [23] to extract call graphs at scale. GraphRAG [4] and its multi-agent extension [8] show that LLMs can reason effectively over graph-structured knowledge, providing architectural precedent for a graph-backed multi-agent loop. None of these uses a code graph as a live write-aware coordination layer.

2.3 Concurrency Control and Collaborative Editing

Ellis and Gibbs [5] introduced Operational Transformation for convergent collaborative text editing. Shapiro et al. [21] formalized CRDTs, guaranteeing strong eventual consistency without locking. Kleppmann et al. [12] extended CRDTs to rich-text editing. All operate at character granularity and make no semantic guarantees.

Pugachev [18] applies CRDTs to multi-agent LLM code generation, achieving 100% syntactic convergence across 600 trials with up to 21.1% speedup on parallelizable tasks. However, a residual 5–10% semantic conflict rate persists despite syntactic convergence. This empirically establishes that text-level consistency is strictly weaker

than the coordination guarantee Kovex targets. Gray et al. [7] analyze lock granularity trade-offs; Kovex adopts an analogous design, operating at semantic node granularity rather than file-level or character-level.

2.4 Verification, Semantic Conflicts, and Interface Contracts

Meyer's Design by Contract [16] established preconditions, postconditions, and invariants as contractual obligations. In a multi-agent setting, a function signature constitutes an implicit contract; a write that alters it without notifying dependents is the formal definition of a semantic conflict. Krieter et al. [13] show that higher-order semantic merge conflicts are common in open-source projects even without concurrent agents. Santos et al. [20] propose lightweight static analysis for post-hoc detection; Barbosa et al. [2] improve recall via pointer analysis. Kovex targets the same failure class but prevents conflicts at coordination time rather than detecting them after the fact. Consumer-driven contract testing [1, 15] provides a structural analogy: a read-registration is equivalent to a consumer contract declaration, and write-triggered notification is the automated provider responsibility mechanism.

3 System Design

3.1 Overview

Kovex comprises three components: the *Kovex Graph* (a Neo4j labeled property graph indexed from live source code), the *MCP Server* (a Model Context Protocol server exposing the graph to agents as a tool API and maintaining a read registry), and the *Write Log* (an append-only record of every write event and its notifications for evaluation).

3.2 Graph Schema

The graph models the *semantic contract surface* of a codebase: named entities whose interface, when changed, can invalidate a dependent agent's code. Four node labels are defined: `Function` (with properties `name`, `file`, `signature`, `return_type`, `body_hash`), `Type` (classes, interfaces, structs), `Module` (source files), and `Symbol`

(module-level variables and constants). Seven edge types encode semantic dependencies: CALLS, USES_TYPE, DEFINES, IMPORTS, INHERITS, IMPLEMENTS, and OVERRIDES. The graph is indexed using Tree-sitter [23] for language-agnostic AST extraction and incremental re-indexing on each write.

3.3 Read/Write State Model

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be the Kovex graph and $\mathcal{A}$ the set of active agents. Each agent a maintains a read set $\mathrm{RS}(a) \subseteq \mathcal{V}$. The global read registry is the relation $\mathcal{R} = \{(a, v, t_r, h_r) \mid v \in \mathrm{RS}(a)\}$, where t_r is the registration timestamp and h_r is the node's `body_hash` at that time. A *write event* by agent a to node v with patch Δ is *observable* iff $h_{\mathrm{post}}(v) \neq h_{\mathrm{pre}}(v)$; non-observable writes (no contract-surface change) do not trigger notifications.

3.4 Write-Triggered Notification Algorithm

Definition 1: k-Hop Notification Radius

Let $d_{\mathcal{G}}(u, v)$ denote the shortest-path distance between nodes u and v in $\mathcal{G}$ restricted to CALLS edges. The *k-hop dependency neighborhood* of v is $\mathcal{N}_k(v) = \{u \in \mathcal{V} \mid d_{\mathcal{G}}(v, u) \leq k\}$. Agent a is a *notification target* for a write to v iff $\mathrm{RS}(a) \cap \mathcal{N}_k(v) \neq \emptyset$.

Edge type selection. A key design decision is which edge types to include in the BFS traversal. An undirected BFS over all edge types (CALLS, USES_TYPE, DEFINES, etc.) places every function pair co-defined in the same module at $d \leq 2$ via DEFINES edges and every pair sharing a parameter type at $d \leq 2$ via USES_TYPE edges. While this minimizes false negatives, it conflates module co-location with semantic dependency, over-notifying agents about writes to unrelated functions. Kovex constrains the BFS to CALLS edges only, ensuring notifications fire strictly on caller/callee relationships.

Algorithm 1 gives the formal procedure.

Algorithm 1 Write-Triggered Notification (NotifyDependents)

Require: Graph $\mathcal{G}$, written node v, hop radius k, read registry $\mathcal{R}$; BFS restricted to Calls edges

Ensure: Notification triples (a, u, d)

```
frontier ← {v};   visited ← {v};   notifs ← ∅
for d = 1 to k do
    next ← ∅
    for each u ∈ frontier do
        for each w ∈ CallNeighbors(u, G) do
            if w ∉ visited then
                visited += w;   next += w
                for each (a, w, t_r, h_r) ∈ R do
                    notifs += (a, w, d)
                end for
            end if
        end for
    end for
    frontier ← next
end for
return notifs
```

3.5 MCP Server Interface

Eight tools are exposed: `register_read` (add entry to $\mathcal{R}$), `deregister_read`, `list_reads`, `commit_write` (apply patch, run Algorithm 1, append to write log, return notify set), `get_node` (ephemeral read; no registry side effect), `get_dependents` (enumerate $\mathcal{N}_k(v)$), `query_graph` (read-only Cypher), and `poll_directives` (retrieve and clear pending verification directives). The distinction between `get_node` and `register_read` is a deliberate interface contract: ephemeral reads do not enroll the agent in the notification protocol.

3.6 Semantic Conflict Definition

Definition 2: Semantic Conflict

Let $\text{Valid}(C, \mathcal{G}_t)$ denote that code fragment C type-checks against codebase state $\mathcal{G}_t$. A **semantic conflict** between agents A and B with respect to node v at time t_w holds iff:

(SC1) Agent A commits an observable write to v at t_w.

(SC2) $(B, u, t_r, h_r) \in \mathcal{R}$ for some $u \in \mathcal{N}_k(v)$, with $t_r < t_w$.

(SC3) Agent B produced code C_B in $[t_r, t_w)$.

(SC4) $\text{Valid}(C_B, \mathcal{G}_{t_w^-}) \wedge \neg\,\text{Valid}(C_B, \mathcal{G}_{t_w^+})$.

Condition (SC4) is operationalized by running `mypy` on C_B against the pre-write and post-write codebase states; a failure attributable to the changed node constitutes a classified conflict.

4 Evaluation Setup

4.1 Codebase and Graph Statistics

We index the httpx Python HTTP client library [17], a production-grade codebase with well-defined cross-file type dependencies. The resulting Kovex graph contains 550 nodes (358 `Function`, 85 `Type`, 84 `Symbol`, 23 `Module`) and 916 edges (527 DEFINES, 225 USES_TYPE, 99 CALLS, 56 INHERITS, 9 IMPORTS).

4.2 Task Design

We design 14 evaluation tasks organized into four groups based on the CALLS-chain distance d between the write target and Agent B's registration node. Each task specifies: a write target (node that Agent A modifies), a registration node (node that Agent B registers a read dependency on), and Agent B's code C_B that depends on the write target's pre-write interface. All tasks use a realistic signature change: parameter removal, parameter renaming, or addition of a required positional parameter.

Tasks at $d = 0$ (T01–T02) test direct read/write conflict on the same node. Tasks at $d = 1$ (T03–T08) test single-hop CALLS-chain notification, covering three write

targets: `AsyncClient.send`, `AsyncClient.request`, `AsyncClient._send_handling_redirects`, and `AsyncClient._send_single_request`. Tasks at $d = 2$ (T09–T12) test two-hop traversal across the call chain `get`/`post` → `request` → `send`. Tasks T13–T14 are *false-negative controls*: the registration node is at $d = 3$ from the write target, beyond the $k = 2$ radius, so Kovex correctly issues no notification despite a real conflict existing.

Table 1 summarizes all 14 tasks.

Table 1: Evaluation tasks. d: CALLS-chain distance from write target to registration node. FN: false-negative control ($d > k$).

Task	Write Target	B Registration	Change Ty
T01	`AsyncClient.send`	`AsyncClient.send`	param remo
T02	`AsyncClient.request`	`AsyncClient.request`	param rena
T03	`AsyncClient.send`	`AsyncClient.request`	param remo
T04	`AsyncClient.send`	`AsyncClient._send_handling_auth`	param remo
T05	`AsyncClient.request`	`AsyncClient.get`	param rena
T06	`AsyncClient.request`	`AsyncClient.post`	param rena
T07	`AsyncClient._send_handling_redirects`	`AsyncClient._send_handling_auth`	param addi
T08	`AsyncClient._send_single_request`	`AsyncClient._send_handling_redirects`	param addi
T09	`AsyncClient.send`	`AsyncClient.get`	param remo
T10	`AsyncClient.send`	`AsyncClient.post`	param remo
T11	`AsyncClient._send_handling_redirects`	`AsyncClient.send`	param addi
T12	`AsyncClient._send_single_request`	`AsyncClient._send_handling_auth`	param addi
T13	`AsyncClient._send_single_request`	`AsyncClient.send`	param addi
T14	`AsyncClient._transport_for_url`	`AsyncClient._send_handling_auth`	param addi

4.3 Experimental Protocol

Each task is executed twice in sequence against a live Neo4j instance and MCP server. In the *Kovex* condition, Agent B calls `register_read` on the registration node before Agent A calls `commit_write` on the write target. In the *baseline* condition, `register_read` is omitted and no directives are issued. After each write, the source patch is applied to the httpx codebase and `mypy` is run on C_B against both pre-write and post-write states to operationalize Definition 2 condition (SC4). The httpx source

and graph node are reset to the canonical pre-write state between tasks to prevent contamination. We measure: (1) notification latency ($t_{\text{directive}} - t_{\text{write}}$ in milliseconds), (2) directive distance accuracy (whether reported d matches expected d), (3) conflict classification accuracy ($\text{Valid}(C_B, \text{pre})$ and $\neg\text{Valid}(C_B, \text{post})$), and (4) notification suppression for $d > k$ tasks.

5 Results

5.1 Coordination Correctness

Table 2 reports per-task quantitative outcomes. All 14 tasks pass every applicable check. In the Kovex condition, all 12 tasks at $d \leq 2$ (T01–T12) receive exactly one directive with the correct distance label, and zero directives are issued for T13 and T14 ($d = 3 > k = 2$). In the baseline condition, zero directives are issued for all 14 tasks and all 14 conflicts remain undetected — a 100% undetected conflict rate without coordination.

Table 2: Per-task results. Kovex: Notified = directive received with correct distance. Baseline: Undetected = conflict exists, zero directives. All tasks satisfy $\text{Valid}(C_B, \text{pre}) = \text{True}$ and $\neg\text{Valid}(C_B, \text{post}) = \text{True}$.

Task	d	**Kovex Notified**	**Distance Correct**	**Latency (ms)**	**Baseline Undetected**	**Conflict Cl**
T01	0	✓	✓	34.0	✓	✓
T02	0	✓	✓	44.3	✓	✓
T03	1	✓	✓	32.5	✓	✓
T04	1	✓	✓	81.8	✓	✓
T05	1	✓	✓	27.9	✓	✓
T06	1	✓	✓	38.3	✓	✓
T07	1	✓	✓	70.6	✓	✓
T08	1	✓	✓	32.6	✓	✓
T09	2	✓	✓	32.9	✓	✓
T10	2	✓	✓	65.5	✓	✓
T11	2	✓	✓	43.7	✓	✓
T12	2	✓	✓	42.9	✓	✓
T13	3	— (FN)	—	—	✓	✓
T14	3	— (FN)	—	—	✓	✓

5.2 Notification Latency

All 12 notifications for T01–T12 arrive within 500 ms, well within the threshold established in the system specification. Table 3 summarizes latency by distance group.

Table 3: Notification latency by distance group ($k = 2$, CALLS-only BFS).

Distance Group	n	**Mean (ms)**	**Range (ms)**	**Std (ms)**
$d = 0$	2	39.2	[34.0, 44.3]	7.3
$d = 1$	6	47.3	[27.9, 81.8]	21.2
$d = 2$	4	46.2	[32.9, 65.5]	14.7
All ($d \leq 2$)	12	45.6	[27.9, 81.8]	17.4

Mean latency does not increase monotonically with d (d=1: 47.3 ms; d=2: 46.2 ms), reflecting that graph traversal cost is dominated by Neo4j query overhead rather than BFS depth for this codebase size. The maximum observed latency of 81.8 ms (T04) occurs at d=1 and is within the 500 ms threshold by a factor of six.

5.3 Conflict Classification

All 14 tasks satisfy Definition 2 conditions (SC1)–(SC4): every write is observable ($h_{\text{pre}} \neq h_{\text{post}}$), every C_B passes `mypy` before the write ($\text{Valid}(C_B, \text{pre}) = \text{True}$), and every C_B fails `mypy` after the write with errors traceable to the changed node ($\neg\text{Valid}(C_B, \text{post}) = \text{True}$). Representative error classes observed: `[call-arg]` for unexpected or missing keyword arguments, `[override]` for subclass method signature incompatibility, and `[name-defined]` for removed parameter names used in callee bodies.

5.4 False-Negative Rate and Sensitivity to k

T13 and T14 lie at $d = 3$ from their respective write targets in the CALLS-only graph. With $k = 2$, Kovex issues zero notifications for both tasks—the write events complete (observable, logged) and conflicts exist (classified by `mypy`), but no directive reaches Agent B. This establishes a false-negative rate of $2/14 = 14.3\%$ at $k = 2$ for this task set. Both false negatives would be resolved by increasing k to 3, which would extend the BFS to include `_send_single_request` → `_send_handling_redirects` →

`_send_handling_auth` → `send` and `_transport_for_url` → `_send_single_request` → `_send_handling_redirects` → `_send_handling_auth`, at the cost of additional false-positive notifications for unaffected agents.

6 Discussion

Edge type selection as a design parameter. An initial implementation of the BFS traversal used all edge types (CALLS, USES_TYPE, DEFINES). Under this configuration, every function pair co-defined in the same source module is reachable at $d \leq 2$ via DEFINES edges, and every pair sharing a parameter type is reachable at $d \leq 2$ via USES_TYPE edges. This eliminates false negatives entirely for same-module writes but conflates module co-location with semantic dependency—notifying agents about writes to unrelated functions. Restricting BFS to CALLS edges restores the intended semantics: notifications fire when a function's callers or callees are registered dependents. This design choice should be made explicit in any deployment: USES_TYPE traversal may be appropriate when type definition changes are the primary coordination concern, while CALLS-only traversal is more precise for function signature changes.

Scope and limitations. The current evaluation is conducted on a single codebase (httpx) with a controlled two-agent scenario and deterministic signature changes. Real multi-agent deployments involve non-deterministic code generation, more than two concurrent agents, and writes that may affect multiple nodes simultaneously. The conflict classification mechanism (mypy) is Python-specific; extension to other languages would require corresponding type checkers. The evaluation does not measure the downstream benefit of receiving a notification—whether agents that receive directives actually re-verify and correct their code—which is a human-in-the-loop or agent-capability question outside the scope of this paper.

Comparison to related work. Kovex addresses the 5–10% residual semantic conflict rate identified by CodeCRDT [18] by operating at the semantic graph layer rather than the text layer. It addresses the global verification overhead of AgentForge [14] by restricting re-verification to agents within the write target's k-hop call neighborhood. The approach is complementary to execution-grounded verification: Kovex issues tar-

geted directives, while AgentForge-style verification can serve as the re-verification mechanism triggered by those directives.

7 Conclusion and Future Work

We presented Kovex, a graph-aware coordination substrate for multi-agent LLM coding systems. The key insight is that the semantic dependency structure of a codebase—represented as a property graph queryable via an MCP server—can serve as a coordination layer: agents register read dependencies on named nodes, and a write to any node triggers targeted verification directives to all registered dependents within a k-hop call-graph radius. Evaluation on 14 tasks against the httpx codebase demonstrates 100% notification accuracy for $d \leq 2$ (mean latency 45.6 ms, max 81.8 ms), 100% conflict classification, and a 14.3% false-negative rate at $d = 3$ for $k = 2$.

Future work includes: (1) extending to multi-language codebases and type checkers; (2) evaluating with live LLM agents to measure downstream conflict prevention; (3) studying the optimal k as a function of codebase topology and agent task distribution; and (4) integrating USES_TYPE traversal for type-definition-centric conflict scenarios.

The Kovex implementation and evaluation harness are available at https://github.com/Ibrahem-al/kovex-research.

Acknowledgments

AI Use Disclosure. The author used Claude (Anthropic) for drafting and editing this manuscript and for code assistance during the implementation of Phases 3 and 4. Perplexity AI was used to assist in identifying and locating relevant academic literature. All scientific claims, formal definitions, algorithmic content, experimental design, and results were reviewed, verified, and approved by the author. No AI system is listed as a co-author or cited as a reference.

References

[1] Aalto University. Consumer-driven contract testing for microservices: Prac-

tical evaluation in a distributed organization. Master's thesis, Aalto University, 2024. https://aaltodoc.aalto.fi/server/api/core/bitstreams/e035e9e7-b7a8-43c2-8c37-8020ae36dfee/content.

[2] M. Barbosa, P. Borba, R. Bonifácio, V. Lira, and G. Santos. The effect of pointer analysis on semantic conflict detection. *arXiv preprint arXiv:2507.20081*, 2025.

[3] A. Chen, Y. Yan, and D. Poshyvanyk. ACER: An AST-based call graph generator framework. *arXiv preprint arXiv:2308.15669*, 2023.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, and J. Larson. From local to global: A graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.

[5] C. A. Ellis and S. J. Gibbs. Concurrency control in groupware systems. In *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*, pages 399–407, 1989. doi: 10.1145/67544.66963.

[6] J. Ferrante, K. J. Ottenstein, and J. D. Warren. The program dependence graph and its use in optimization. *ACM Transactions on Programming Languages and Systems*, 9(3):319–349, 1987.

[7] J. Gray, R. A. Lorie, G. R. Putzolu, and I. L. Traiger. Granularity of locks in a shared data base. In *IFIP Working Conference on Modelling in Data Base Management Systems*, pages 428–451, 1975.

[8] A. Gusarov, A. Volkova, V. Khrulkov, A. Kuznetsov, E. Maslov, and I. Oseledets. Multi-agent GraphRAG: A text-to-Cypher framework for labeled property graphs. *arXiv preprint arXiv:2511.08274*, 2025.

[9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *International Conference on Learning Representations*, 2024. arXiv:2308.00352.

[10] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. L. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.

[11] A. Khatua, H. Zhu, P. Tran, A. Prabhudesai, F. Sadrieh, J. K. Lieberwirth, X. Yu, Y. Fu, M. J. Ryan, J. Pei, and D. Yang. CooperBench: Why coding agents cannot be your teammates yet. *arXiv preprint arXiv:2601.13295*, 2026.

[12] M. Kleppmann, A. Wiggins, P. van Hardenberg, and M. McGranaghan. Peritext: A CRDT for collaborative rich text editing. *Proceedings of the ACM on Human-Computer Interaction*, 5(CSCW2), 2021. https://www.inkandswitch.com/peritext/static/cscw-publication.pdf.

[13] S. Krieter, J. Garcia, S. Apel, et al. Detecting higher-order merge conflicts in large software projects. In *IEEE International Conference on Software Testing, Verification and Validation*, 2020. https://aip.ifi.uni-heidelberg.de/fileadmin/papers/2020/ICST2020-merge-conflicts-final.pdf.

[14] R. Kumar, W. Ali, J. Ahmed, N. I. Ali, and S. Usman. AgentForge: Execution-grounded multi-agent LLM framework for autonomous software engineering. *arXiv preprint arXiv:2604.13120*, 2026.

[15] J. Lehvä, N. Mäkitalo, and T. Mikkonen. Consumer-driven contract tests for microservices: A case study. In *Proceedings of JAX London*, 2019. https://helda.helsinki.fi/server/api/core/bitstreams/8923de60-ede7-4b32-a31c-46c129f8197a/content.

[16] B. Meyer. Applying "design by contract". *Computer*, 25(10):40–51, 1992.

[17] E. OSS. httpx: A next generation HTTP client for Python, 2021. URL https://www.python-httpx.org/. https://github.com/encode/httpx.

[18] S. Pugachev. CodeCRDT: Observation-driven coordination for multi-agent LLM code generation. *arXiv preprint arXiv:2510.18893*, 2025.

[19] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun. Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*, 2024. arXiv:2307.07924.

[20] G. Santos, P. Borba, R. Bonifácio, and M. B. de Oliveira. A lightweight technique for detecting semantic conflicts with static analysis. *SSRN preprint*, 2024. doi: 10.2139/ssrn.5035182.

[21] M. Shapiro, N. Preguiça, C. Baquero, and M. Zawirski. Conflict-free replicated data types. Technical Report RR-7506, INRIA, 2011. Extended version published in *Journal of Parallel and Distributed Computing*, 2014. https://doi.org/10.1016/j.jpdc.2014.01.012.

[22] M. V. T. Thai, T. Le, D. N. Manh, H. P. Nhat, and N. D. Q. Bui. SWE-EVO: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.

[23] I. Utkin, E. Spirin, E. Bogomolov, and T. Bryksin. Evaluating the impact of source code parsers on ML4SE models. *arXiv preprint arXiv:2206.08713*, 2022.

[24] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023.

[25] F. Yamaguchi, N. Golde, D. Arp, and K. Rieck. Modeling and discovering vulnerabilities with code property graphs. In *IEEE Symposium on Security and Privacy*, pages 590–604, 2014. doi: 10.1109/SP.2014.44.

[26] J. Yang, A. Wettig, S. Yao, K. Narasimhan, and O. Press. SWE-agent: Agent-computer interfaces enable automated software engineering. *arXiv preprint arXiv:2405.15793*, 2024.